

计算机体系结构

L04 分支预测

计算机体系结构

背景知识 (自学) ③

存储层次 ④

单处理器 (指令级并行) ④

多处理器 (线程级并行) ⑤

加速器 (数据并行) ③

简单流水线 ④

复杂流水线 ②

分支预测

控制相关

分支预测

预测错误恢复

静态预测方法

动态预测方法

Last Time Predictor

2 bits 饱和计数器

两层预测器

混合预测器

指令并行高级技术 ②

控制相关

(Control Dependancy)

控制相关

□ 如果本周期取到的指令是非控制流指令:

- ✓ 下一个周期的PC值应该是下一条顺序指令的地址
- ✓ 已知指令长度的情况下很容易计算

□ 如果本周期取到的指令是控制流指令:

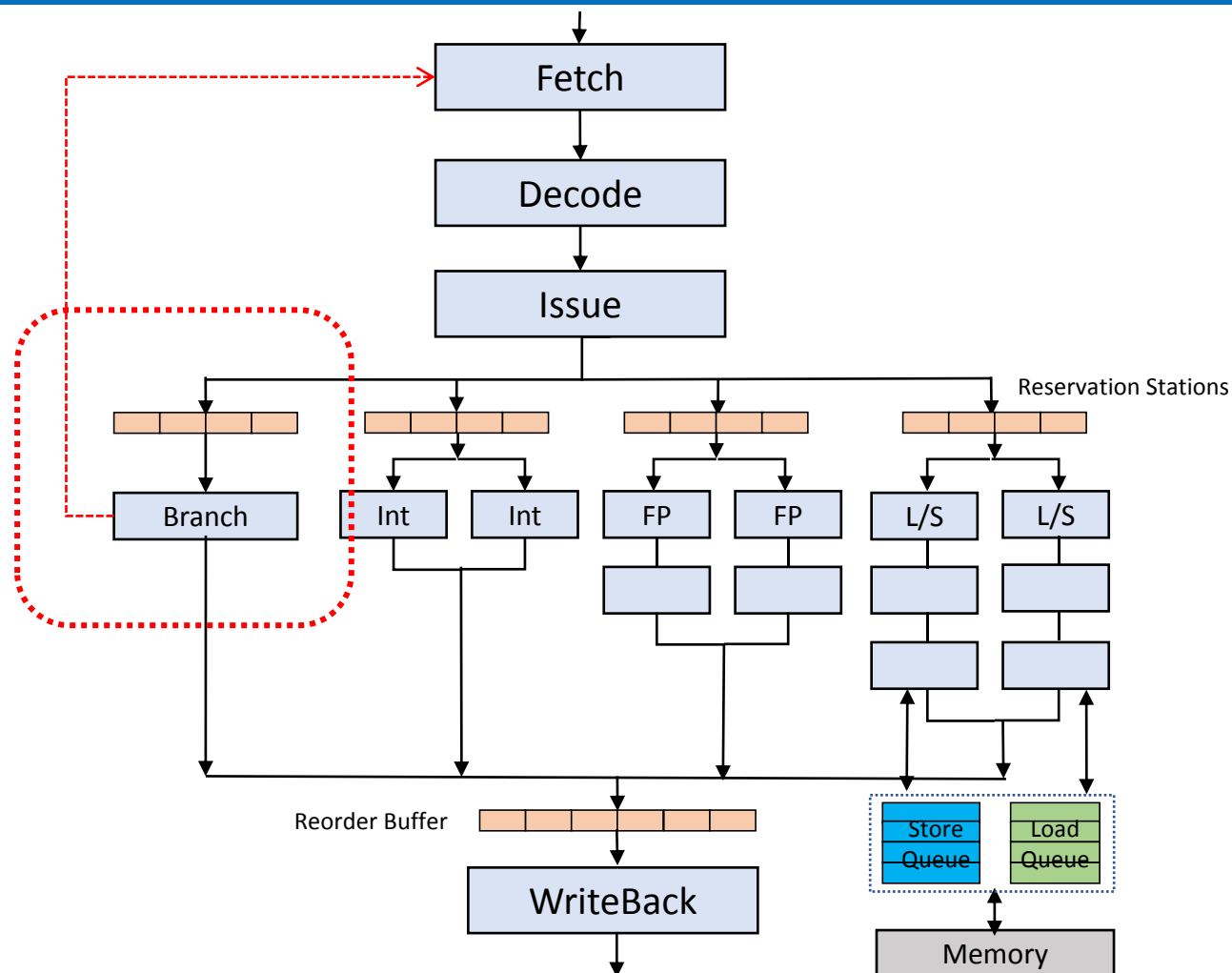
- ✓ 如何决定下一个周期的PC?

□ 我们如何知道获取的指令是否是控制流指令呢？

分支类型

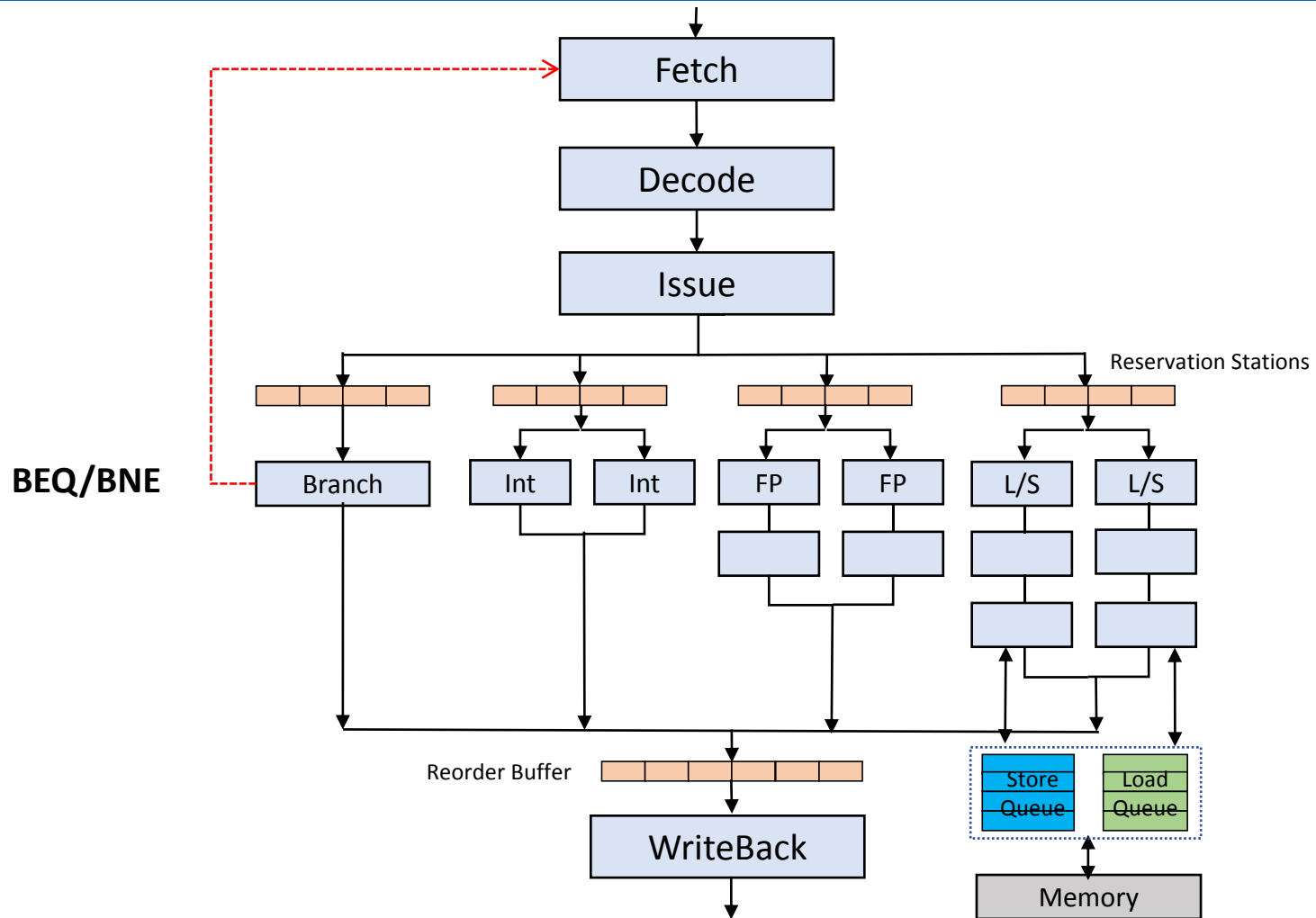
类型	是否跳转	目标地址何时计算出来？
条件分支 beq \$rs, \$rt, offset	X阶段	X阶段 (PC + offset)
无条件分支 J target	Decode阶段	D阶段 (PC高位和target组合)
函数调用 JAL target	Decode阶段	D阶段 (PC高位和target组合)
跳转寄存器 JR \$ra	Decode阶段	Issue阶段 (需要读寄存器)

考虑分支语句的流水线



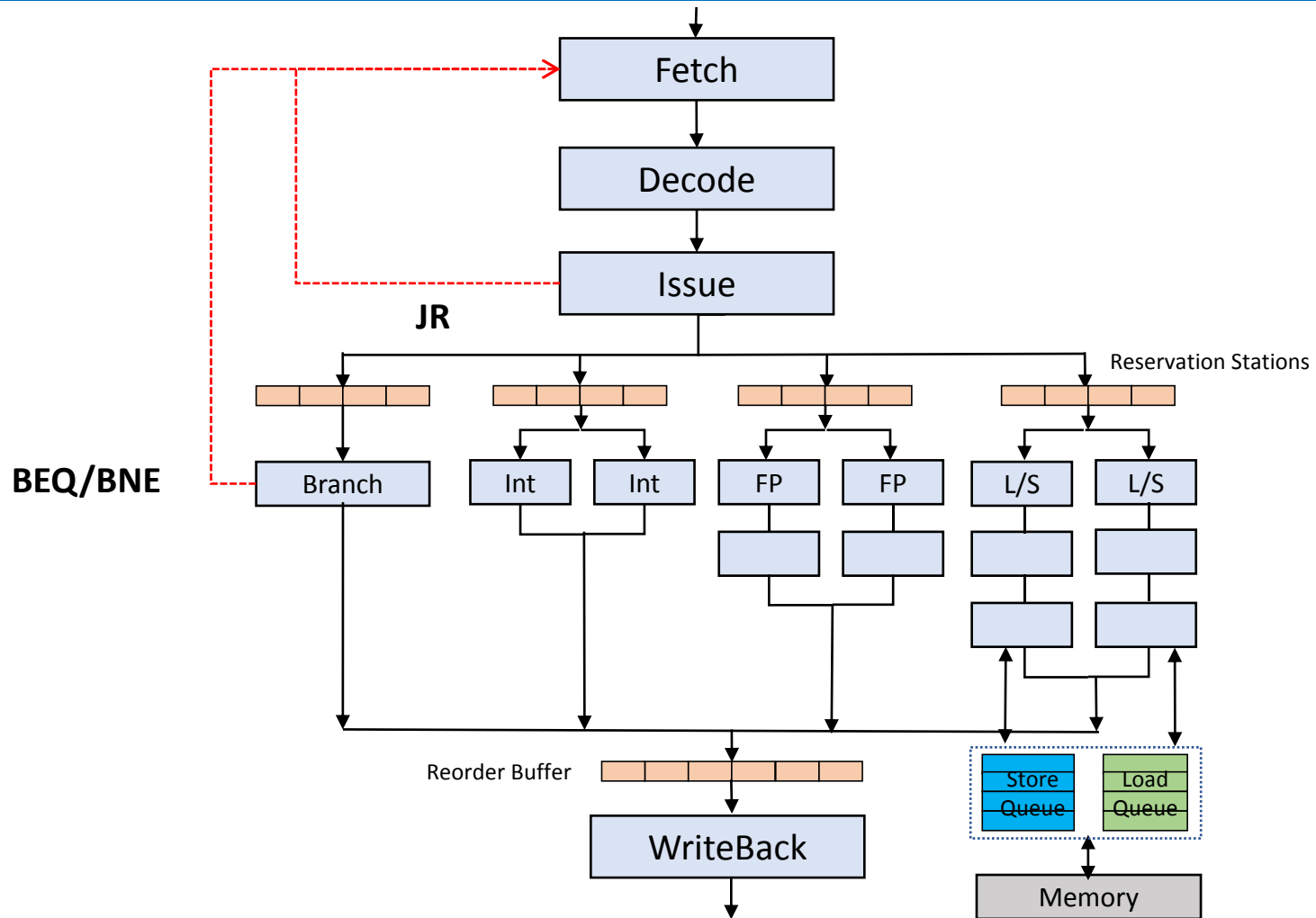
考虑分支的流水线：增加Branch解析单元

考虑分支语句的流水线



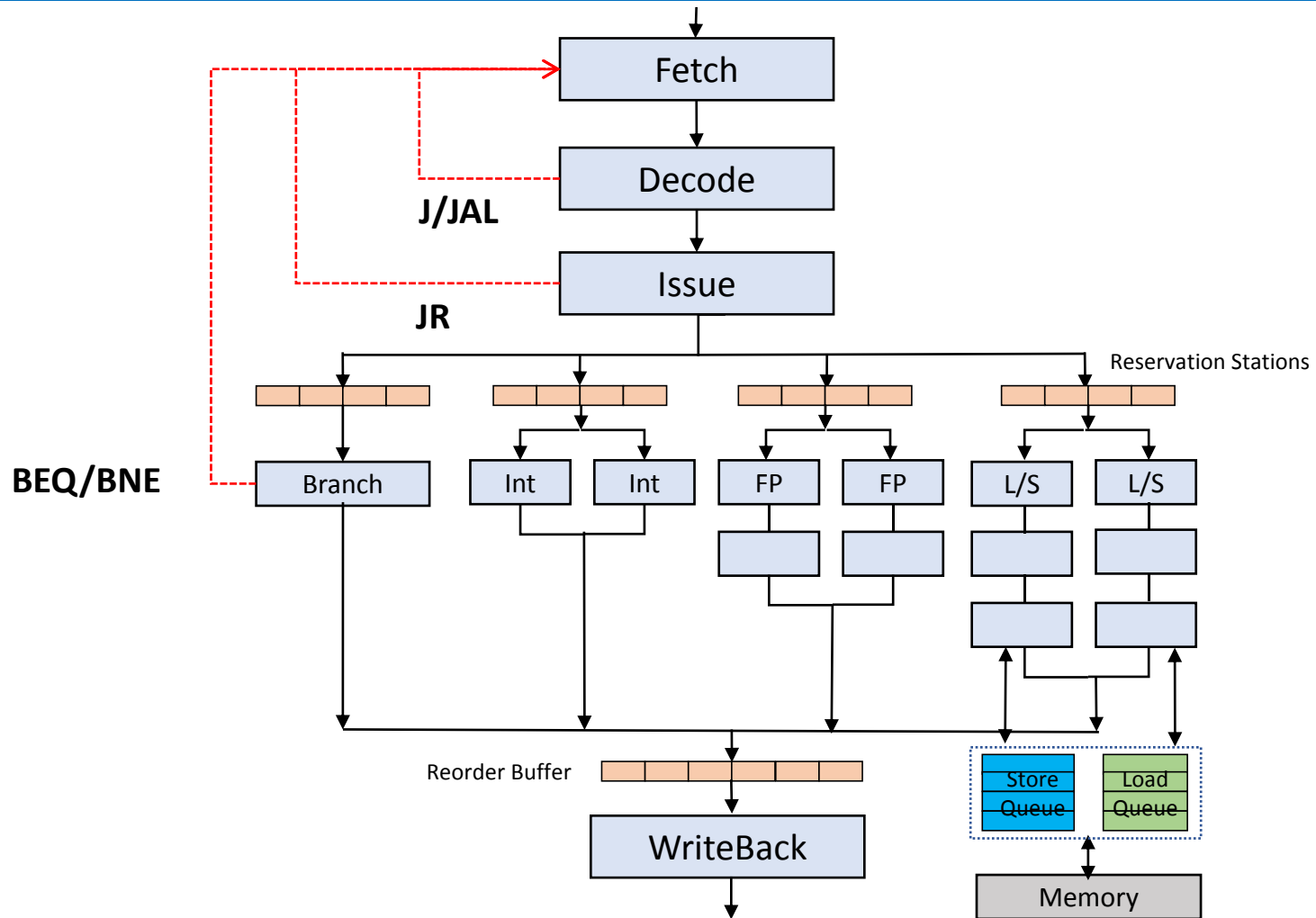
BEQ/BNE指令结果在Execute阶段解析出来

考虑分支语句的流水线



JR指令结果在Issue阶段解析出来

考虑分支语句的流水线



J/JAL指令结果在Decode阶段解析出来

分支语句的占比

SPEC CPU 2017中分支语句的平均占比

	SPECint	SPECfp
Branches	19 %	11 %
Loads	24 %	26 %
Stores	10 %	7 %
Other	47 %	56 %

SPECint17: *perlbench, gcc, mcf, omnetpp, xalancbmk, x264, deepsjeng, leela, exchange2, xz*

SPECfp17: *bwaves, cactus, lbm, wrf, pop2, imagick, nab, fotonik3d, roms*

平均每5-10条指令就有一条分支指令

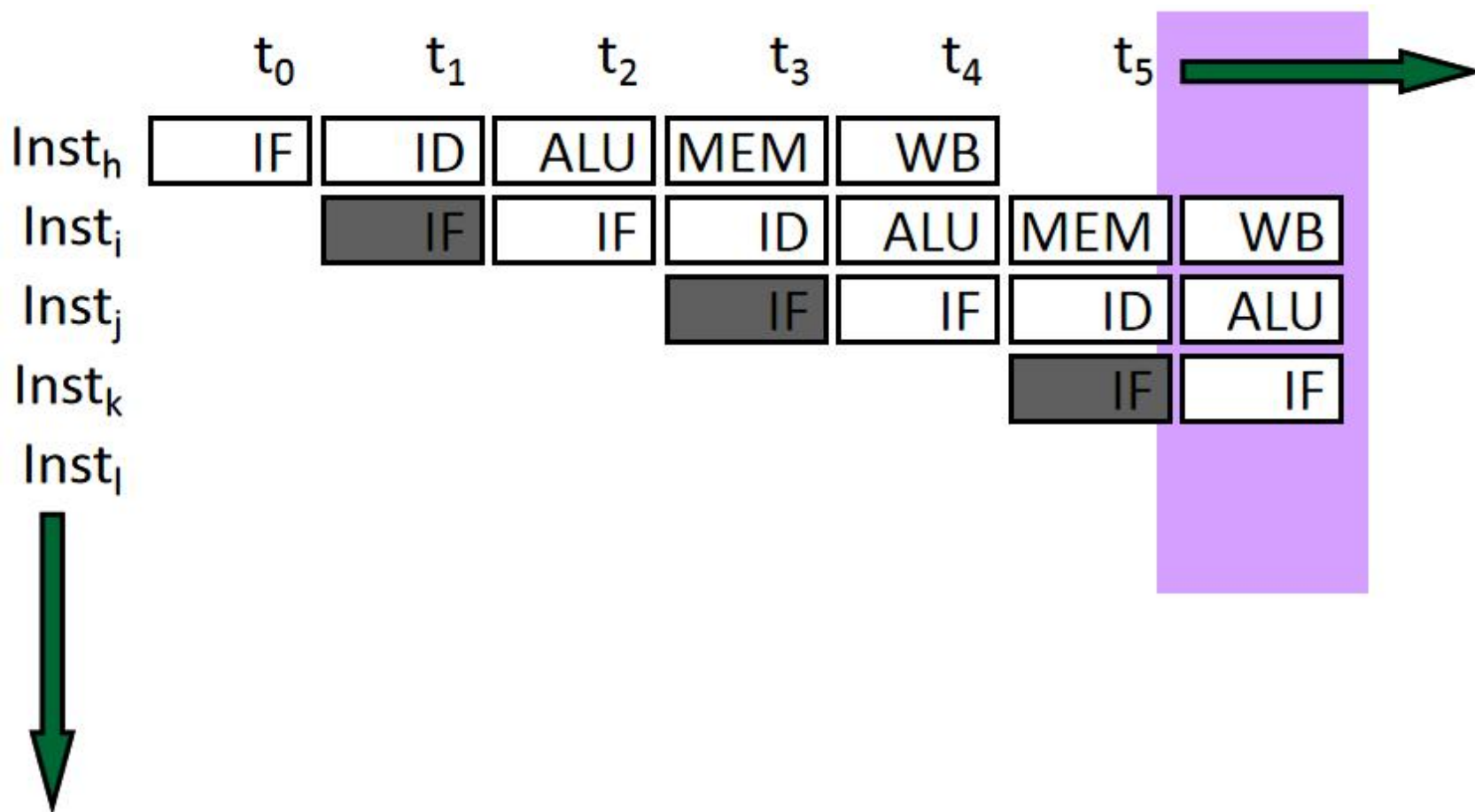
分支预测

(Branch Prediction)

如何处理控制相关

- Stall 流水线直到计算出下一条指令地址
- 猜测下一条指令地址 (分支预测)
- 做些其他事情 (通过多线程掩盖分支条件计算时间)

Stall 流水线是否可行？

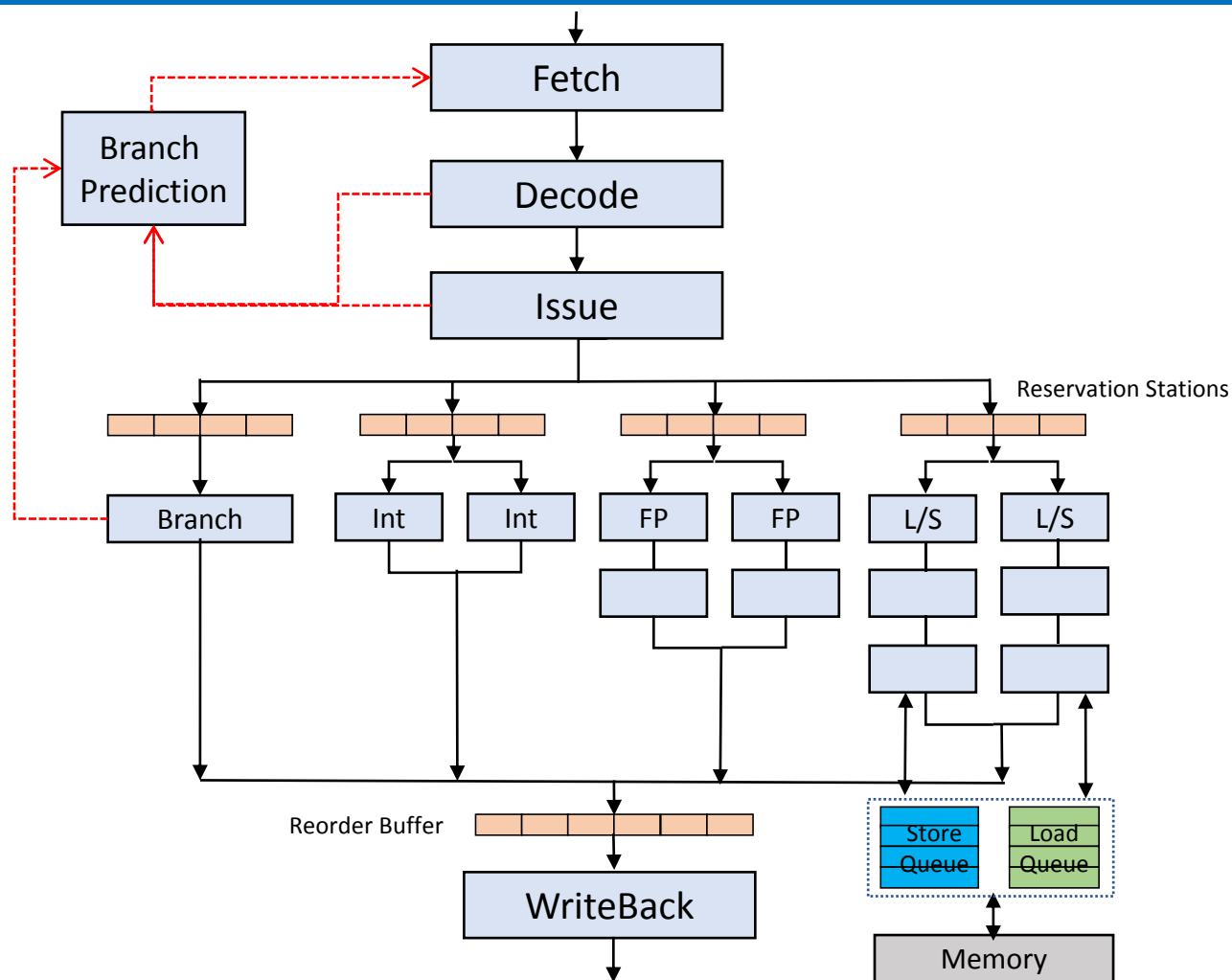


在不考虑分支语句的情况下，每条指令仍需要stall至少1个周期才能被取出！

如何处理控制相关

- ❑ Stall 流水线直到计算出下一条指令地址
- ❑ 猜测下一条指令地址 (分支预测)
- ❑ 做些其他事情 (通过多线程掩盖分支条件计算时间)

带有分支预测的流水线



在流水线中增加分支预测单元

分支预测

无预测：

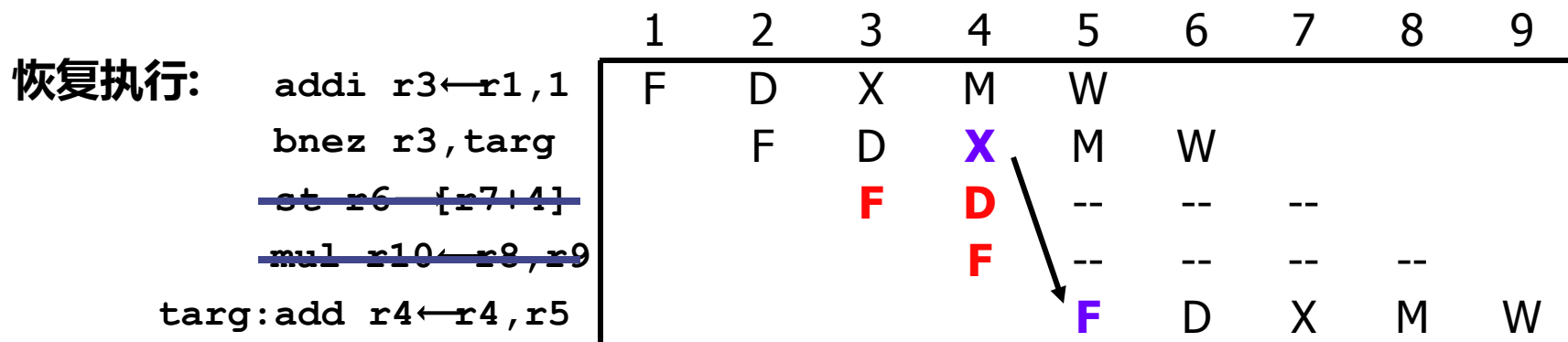
	1	2	3	4	5	6	7	8	9
addi r3←r1,1	F	D	X	M	W				
bnez r3,targ		F	D	X	M	W			
stall			--	--	--	--	--		
stall				--	--	--	--	--	
st r6←[r7+4]					F	D	X	M	W
mul r10←r8,r9									
targ:add r4←r4,r5									

正确预测（预测为不跳转）：

	1	2	3	4	5	6	7	8	9
addi r3←r1,1	F	D	X	M	W				
bnez r3,targ		F	D	X	M	W			
st r6←[r7+4]			F	D	X	M	W		
mul r10←r8,r9				F	D	X	M	W	
				speculative					

正确预测可以节省2个周期！

预测错误的性能损失(Penalty)



□ 预测错误后的恢复过程:

- 对于不太深、顺序执行的流水线没太大问题
- 分支结果在X阶段计算出来
- + 前两条指令 (in F, D) 尚未更改系统状态 (写寄存器/内存)
- **Flush 掉在D和X阶段的指令** (i.e., 替换为**nops**)

性能分析

□ 正确预测 ⇒ 没有penalty

假设准确率86%

□ 预测错误 ⇒ 耽误2个周期

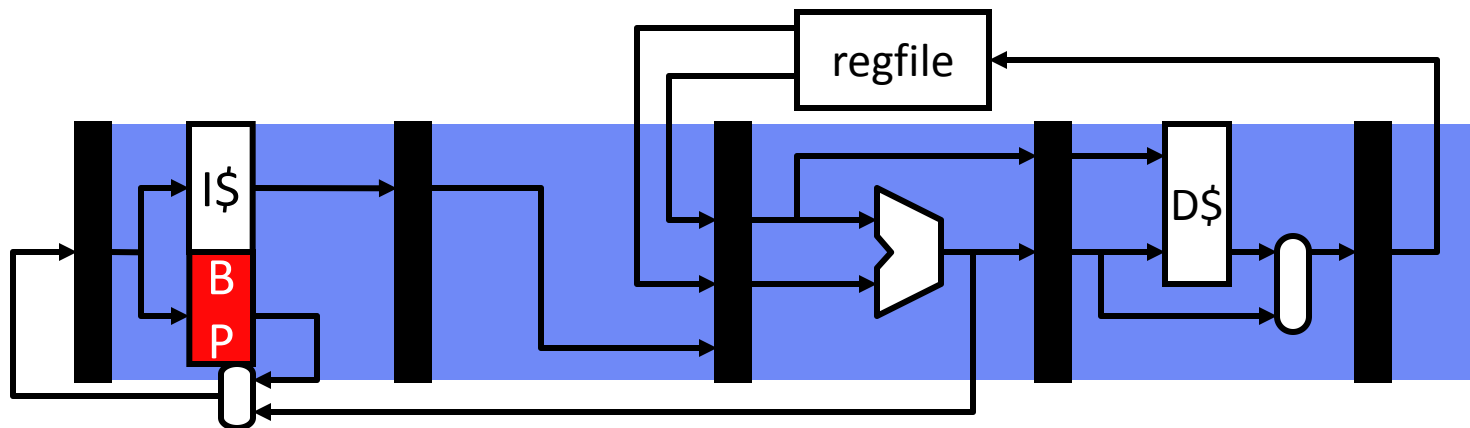
□ 假设

- 无数据危害引起的stalls
- 20% 的分支指令
- 70% 的分支指令发生了跳转（预测的是不跳转，直接取下一条）
- $CPI = [1 + (0.20 * 0.7) * 2] =$
 $= [1 + 0.14 * 2] = 1.28$

预测错误的概率 预测错误的penalty

这两项哪个可以降低？

分支预测的任务



Step #1: 是不是一个分支?

- 在decode阶段很容易判断...

Step #2: 该分支是否跳转?

- 跳转方向预测器 (仅适用于条件分支语句)
- 预测跳转/不跳转

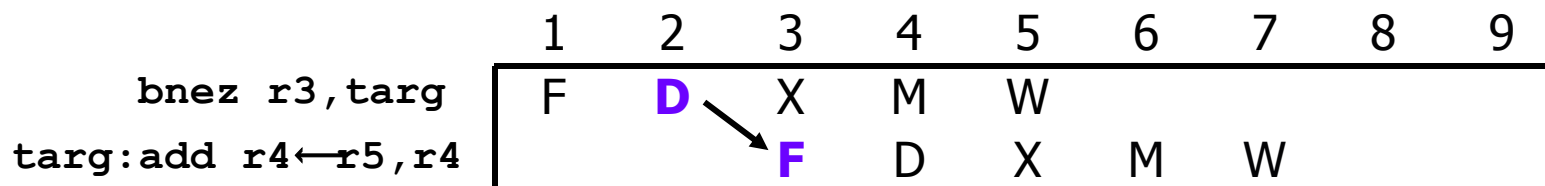
Step #3: 如果跳转, 跳转到哪里?

- 在decode阶段很容易判断...

何时进行预测?

Option #1: 在Decode阶段

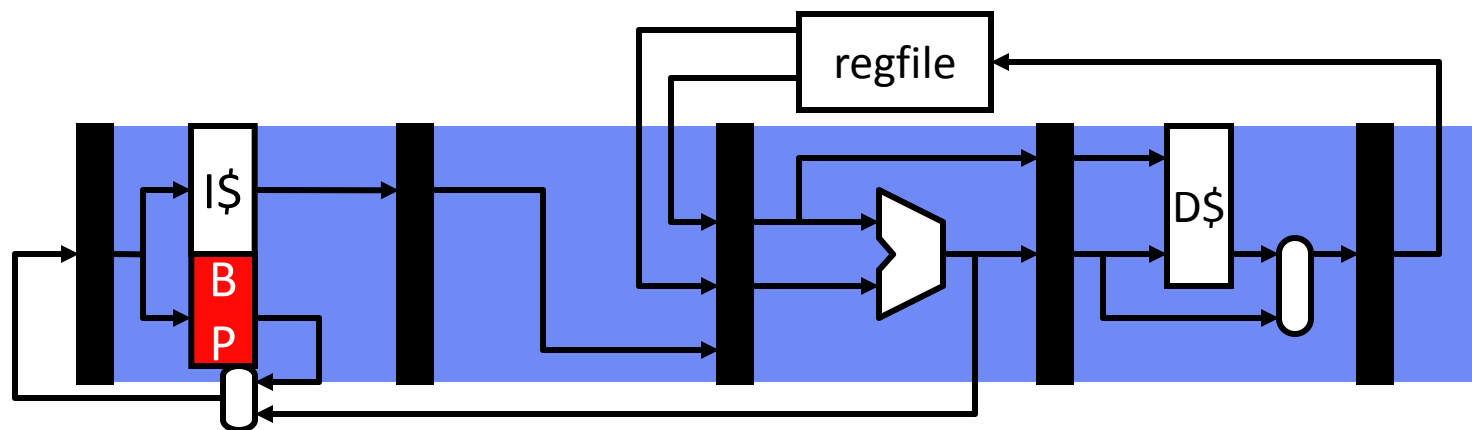
- 解析指令的opcode来判断是否为分支
- 根据预测结果以及解析结果计算下一个PC值 (如果是PC-relative的分支语句)
- 即使预测正确, 仍然会stall一个周期



Option #2: 在Fetch阶段?

- 怎么做?

分支预测的任务



Step #1: 是不是一个分支?

- 在decode阶段很容易判断...如果在fetch阶段，需要预测

Step #2: 该分支是否跳转?

- 跳转方向预测器 (仅适用于条件分支语句)
- 预测跳转/不跳转

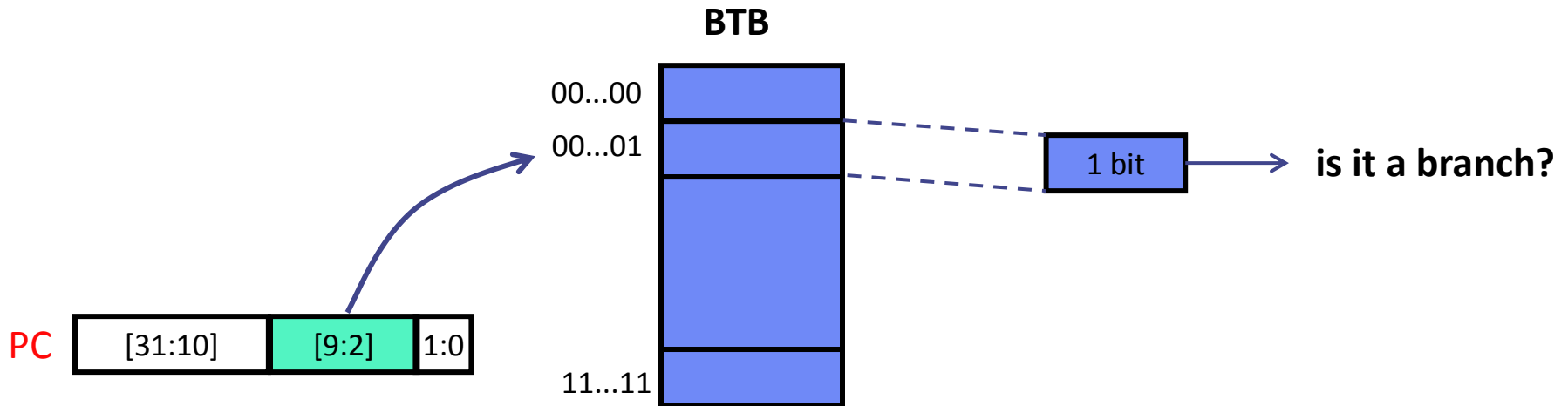
Step #3: 如果跳转, 跳转到哪里?

- 在decode阶段很容易判断...

分支目标缓存 (Branch Target Buffer)

□ Branch target buffer (BTB):

- 记录之前跳转过的一些分支语句
 - 发现跳转的分支语句，放入BTB表
- BTB表由PC的低位进行索引
 - 表的每一项为1 bit: 表示该地址是否为分支语句?
- 映射冲突 (aliasing) ?
 - 两个不同PC的低位相同



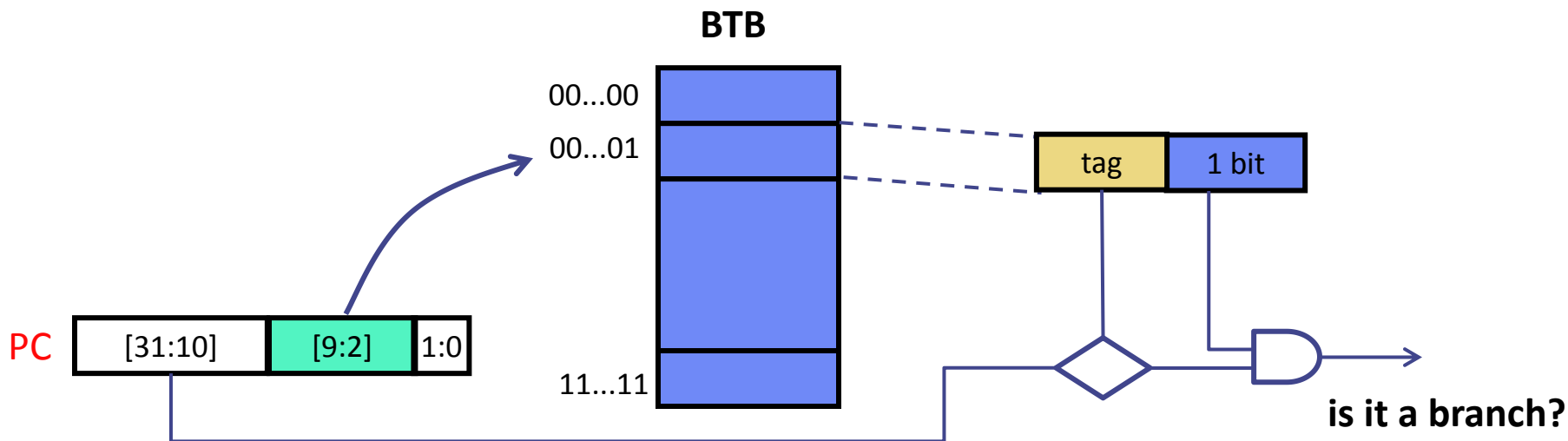
分支目标缓存 (Branch Target Buffer)

□ BTB 表项粒度太粗

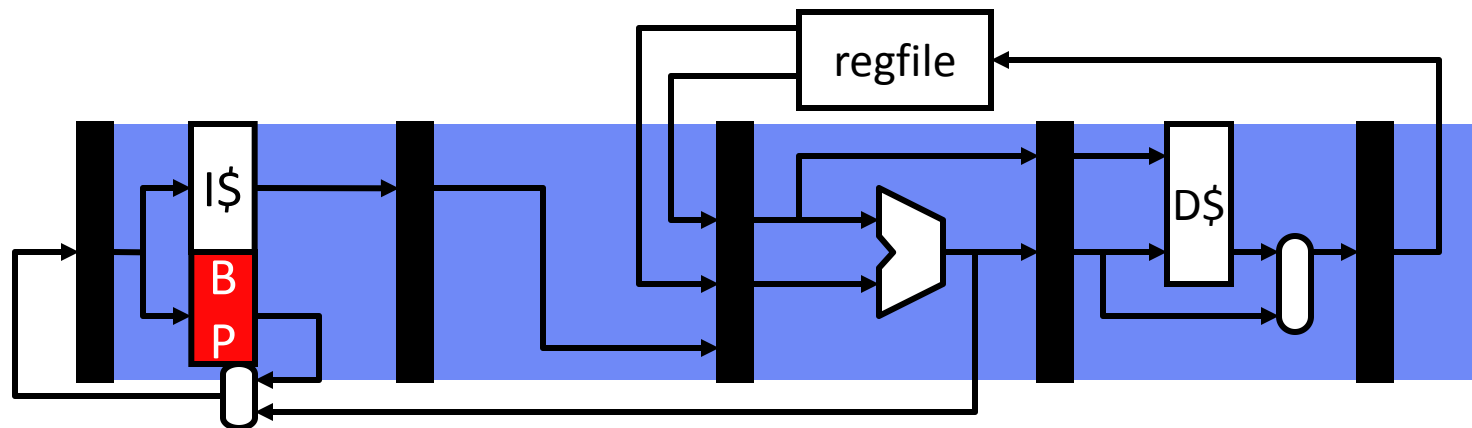
- 映射冲突较多
- 非跳转语句也可能预测为跳转指令

□ 改进：为每个BTB表项增加 **Tag**

- 只有真正的分支语句才能预测为分支语句
- 判断条件：(BTB[PC].branch && BTB[PC].tag == PC)



分支预测的任务



Step #1: 是不是一个分支?

- 在decode阶段很容易判断..

Step #2: 该分支是否跳转?

- 跳转方向预测器 (仅适用于条件分支语句)
- 预测跳转/不跳转

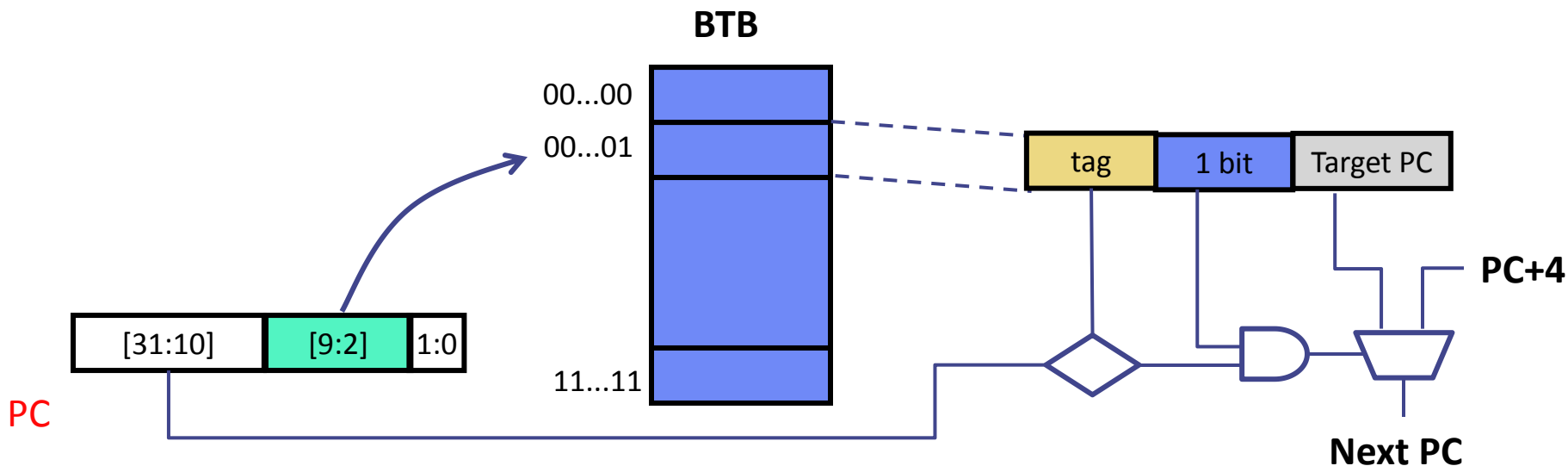
Step #3: 如果跳转, 跳转到哪里?

- 在decode阶段很容易判断...
- 使用BTB表记录目标地址

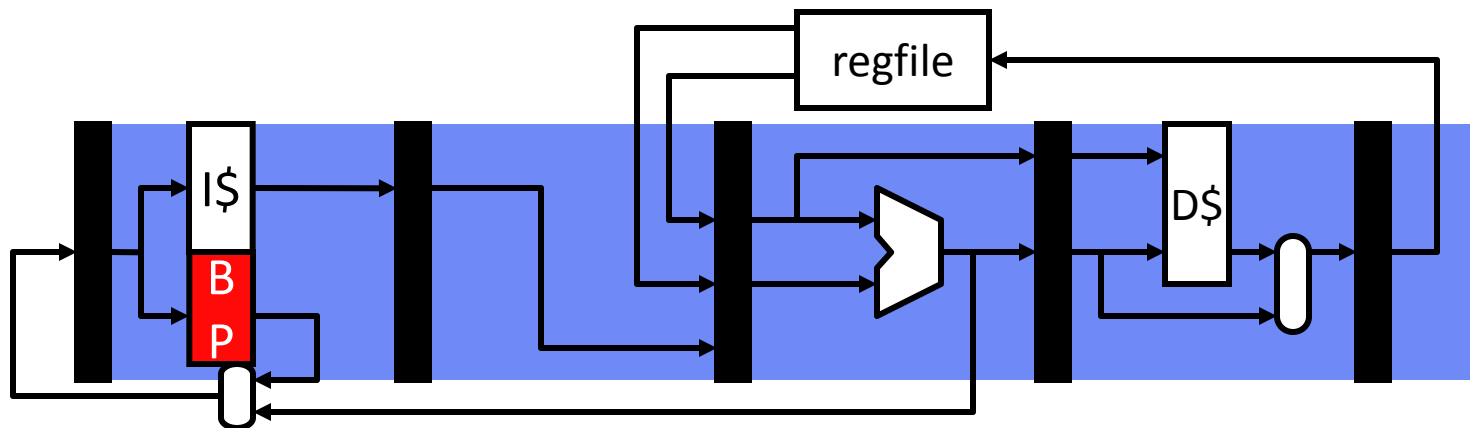
分支目标缓存 (Branch Target Buffer)

❑ Branch target buffer (BTB):

- 在BTB的每一个表项增加Target PC
 - 用来记录上次本条分支语句跳转地址
- 如果分支语句X上一次跳转到了Y
 - 将BTB表项中的Target PC更新为Y
 - 分支语句的跳转地址很大概率和上次相同！



分支预测的任务



Step #1: 是不是一个分支?

- 在decode阶段很容易判断...

Step #2: 该分支是否跳转?

- 跳转方向预测器

Step #3: 如果跳转, 跳转到哪里?

- 在decode阶段很容易判断...

简单分支预测方法

□ 编译时 (静态方法)

- Always not taken
- Always taken
- BTFN (Backward taken, forward not taken)
- Profile based (likely direction)
- Program analysis based (likely direction)

□ 运行时 (动态方法)

- Last time prediction (single-bit)
- Two-bit counter based prediction
- Two-level prediction (global vs. local)
- Hybrid
- Advanced algorithms (e.g., using perceptrons)

静态预测方法 (I)

□ Always not-taken (预测不跳转)

- 实现简单: 不需要BTB, 不需要预测跳转方向
- 准确率低: ~30-40% (对于条件分支)

□ Always taken (预测跳转)

- 不需要预测跳转方向
- 准确率较高: ~60-70% (对于条件分支)
 - ✓ 后向分支 (Backward branche) (i.e. 循环语句)通常会跳转
 - ✓ 后向分支: 跳转的目标地址低于当前PC (往回跳)

□ Backward taken, forward not taken (后向跳转, 前向不跳转, BTFN)

- 预测后向分支语句为跳转,其余分支语句为不跳转

□ Profile-based (基于提前分析)

- 思路: 编译器提前执行程序 (profile run) , 根据执行中的分支跳转情况进行预测, 并将预测结果通过编码提示位编译到代码中

静态预测方法 (II)

□ Program-based (基于程序分析的方法)

- 思路: 通过分析各种指令在各种程序中的表现, 发现一些跳转规律
- 例如: 预测BLEZ为不跳转 (很多程序中小于0的数表示错误信息, 很少出现)
- 例如: 预测循环中的分支语句为跳转
- 如果是指针或者小数进行比较, 预测为不相等

- + 不需要提前profiling
- 有些规律可能不正确
- 需要编译器进行分析, 以及ISA的支持

静态预测方法 (III)

□ Programmer-based (基于程序员的方法)

- 思路: 程序员提供预测信息
- 通过编译提示 (pragmas) 在程序中标记分支语句的跳转可能性

- + 不需要profiling和程序分析
- + 程序员对程序更熟悉，预测结果可能更准确
- 需要编程语言，编译器，ISA的支持
- 增加了程序员的负担？

编译提示 (Pragma)

□ 能够让程序员向底层传递提示信息的关键字

```
if (likely(x)) { ... }
```

```
if (unlikely(error)) { ... }
```

□ 其他例子

- E.g., 是否要对一个循环并行化
- **#pragma omp parallel**

静态预测方法

□ 所有静态方法可以组合

- Profile based
- Program based
- Programmer based

□ 静态预测方法的缺点？

- 无法适应分支行为的动态变化
 - 比如某些分支的行为与运行时的输入数据相关

简单分支预测方法

□ 编译时 (静态方法)

- Always not taken
- Always taken
- BTFN (Backward taken, forward not taken)
- Profile based (likely direction)
- Program analysis based (likely direction)

□ 运行时 (动态方法)

- Last time prediction (single-bit)
- Two-bit counter based prediction
- Two-level prediction (global vs. local)
- Hybrid
- Advanced algorithms (e.g., using perceptrons)

动态分支预测

□ **思路:** 根据运行时信息进行分支预测

□ **优点**

- + 能够适应分支行为的动态变化
- + 不需要静态分析

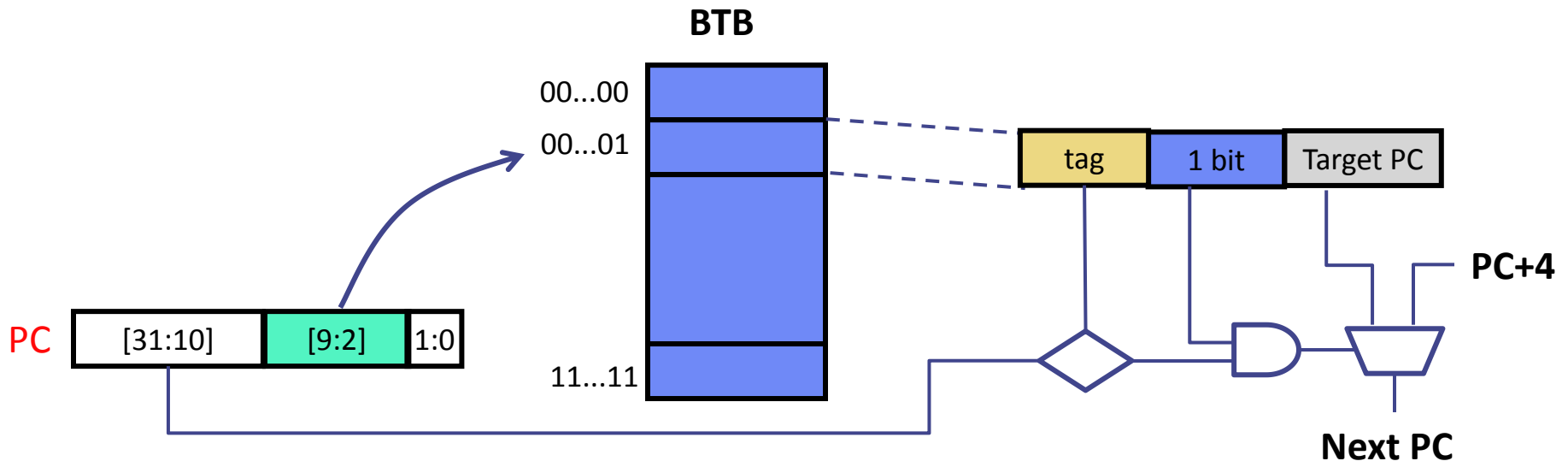
□ **缺点**

- 更复杂 (需要额外的硬件)

Last Time Predictor

□ Last time predictor

- 每个分支语句使用1个bit (stored in BTB)
 - 1 bit 记录预测结果 (0 = N, 1 = T)
 - 也就是上次该分支语句的跳转情况 (N表示不跳转, T表示跳转)
- TTTTTTTTTTNNNNNNNNNNNN → 90% accuracy



Last Time Predictor

□ 问题：双层循环的情况

```
for (i=0;i<100;i++)  
    for (j=0;j<3;j++)  
        // whatever
```

- 每次内循环都会导致2次预测错误
- 预测器状态转换太快了

Time	State	Prediction	Outcome	Result?
1	N	N	T	Wrong
2	T	T	T	Correct
3	T	T	T	Correct
4	T	T	N	Wrong
5	N	N	T	Wrong
6	T	T	T	Correct
7	T	T	T	Correct
8	T	T	N	Wrong
9	N	N	T	Wrong
10	T	T	T	Correct
11	T	T	T	Correct
12	T	T	N	Wrong

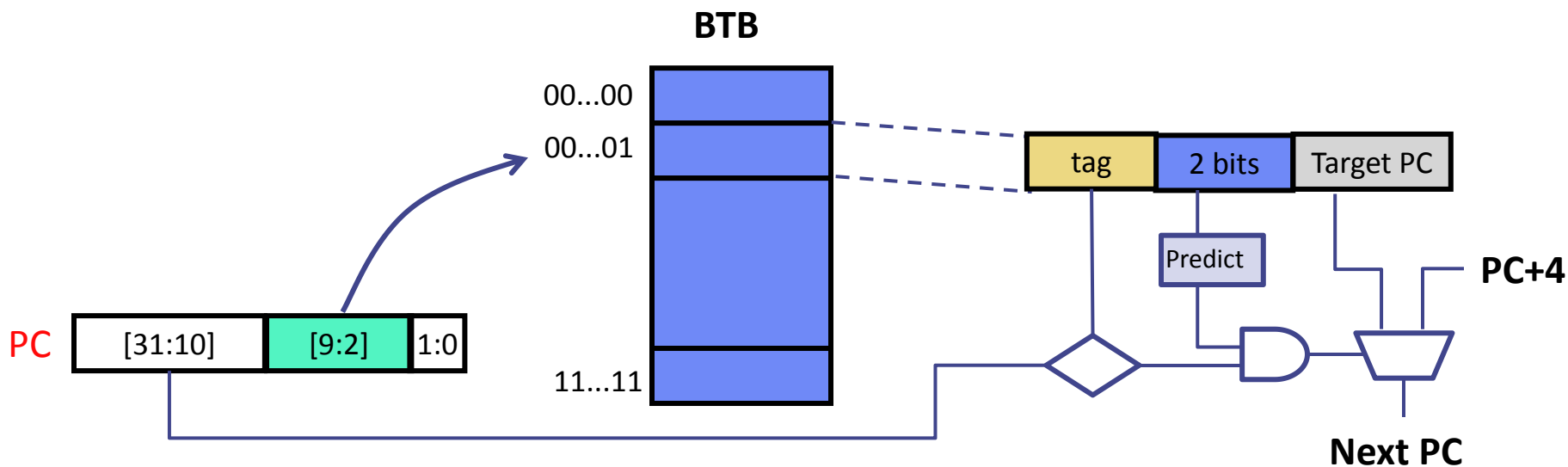
Last Time Predictor 改进思路

- **问题:** last-time predictor 状态变化太快 (T->NT或NT->T)
 - 有些时候分支语句可能是mostly taken 或 mostly not taken
- **改进思路:** 为预测器添加滞后特性，这样在出现单次不同的结果时预测不会立即改变
 - 使用2 bits 记录预测历史
 - T和NT都分别有2个状态表示

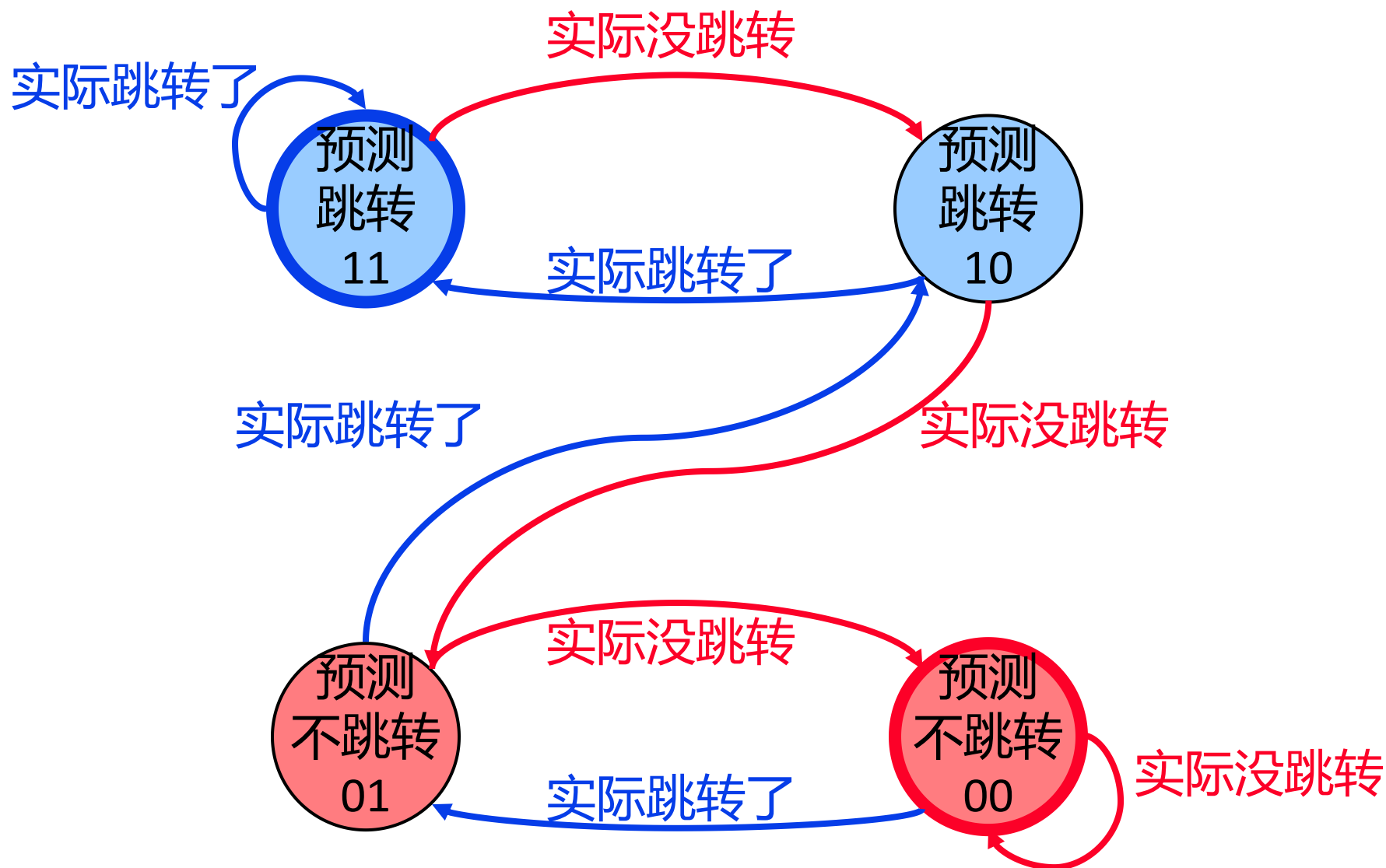
Smith, "A Study of Branch Prediction Strategies," ISCA 1981.

2 bits 饱和计数器

- 每个分支语句用2 bits计数器进行预测
- 2 bits:** (00->N, 01->n, 10->t, 11->T)



2 bits 饱和计数器



2 bits 饱和计数器

```
for (i=0;i<100;i++)  
    for (j=0;j<3;j++)  
        // whatever
```

- 稳定之后每次循环只有1次预测错误

+ 预测准确率更高
-- 硬件开销更大

2bits饱和计数器的预测准确率在85-90%

Time	State	Prediction	Outcome	Result?
1	N	N	T	Wrong
2	n	N	T	Wrong
3	t	T	T	Correct
4	T	T	N	Wrong
5	t	T	T	Correct
6	T	T	T	Correct
7	T	T	T	Correct
8	T	T	N	Wrong
9	t	T	T	Correct
10	T	T	T	Correct
11	T	T	T	Correct
12	T	T	N	Wrong

两层预测器 (Two-Level Prediction)

- 无论是last time predictor还是2bits饱和计数器，都仅利用了分支语句上一次 (last time) 的结果
- 现象 1: 一条分支语句的结果可能与其他分支语句的结果相关
 - 全局相关性
- 现象2: 一条分支语句的结果可能与其过去的很多次结果相关 (不仅仅是最后一次)
 - 局部相关性

全局相关性-例子1

- 如果第一条分支不跳转，第二条也不会跳转

```
if (cond1)
...
if (cond1 AND cond2)
```

- 如果第一条分支跳转，第二条不会跳转

```
branch Y: if (cond1) a = 2;
...
branch X: if (a == 0)
```

全局相关性-例子2

- 如果Y和Z跳转，X也会跳转
- 如果Y或者Z不跳转，X也不会跳转

```
branch Y: if (cond1)
...
branch Z: if (cond2)
...
branch X: if (cond1 AND
cond2)
```

如何利用全局相关性？

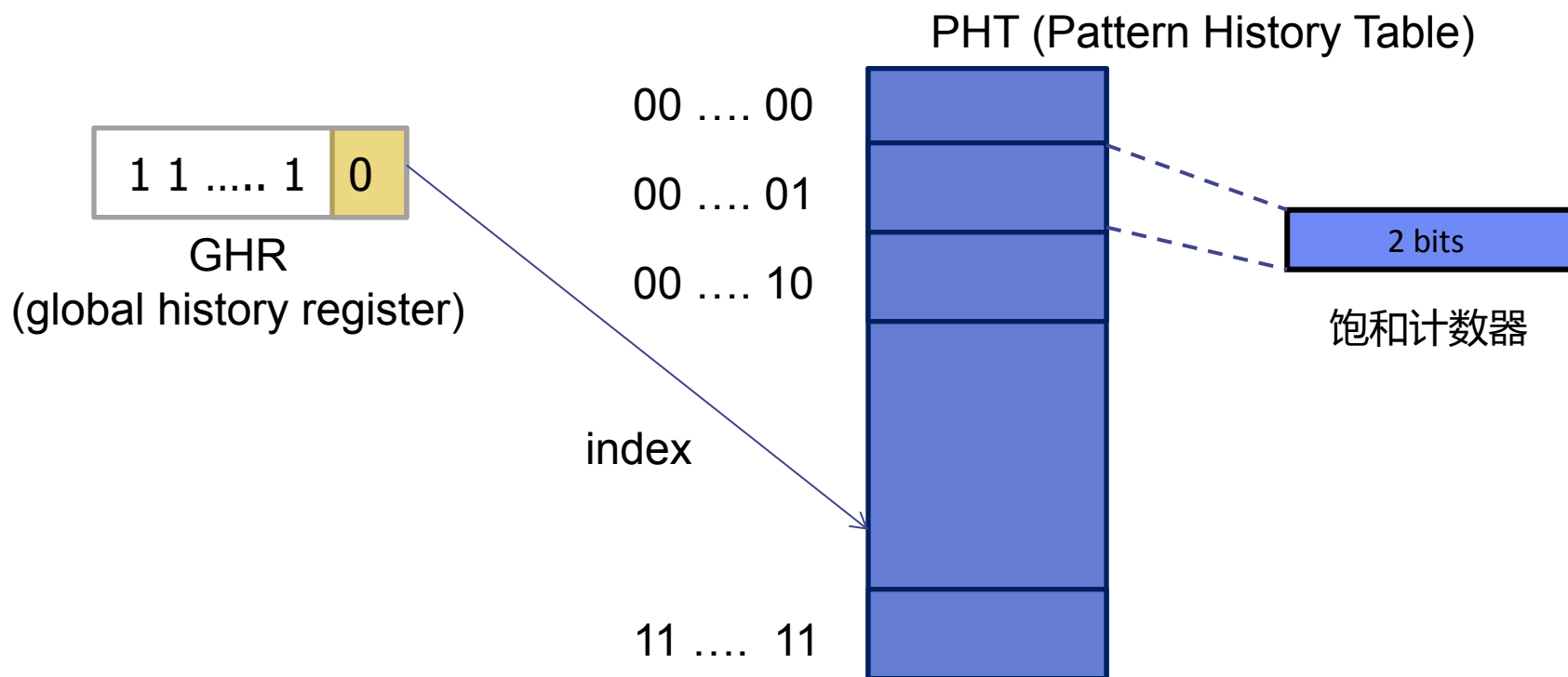
□ **思路:** 记录所有分支语句的历史信息，当历史信息出现某种pattern时，按照上次出现这种pattern的结果进行预测

□ **实现：**

- 利用**全局历史寄存器**（Global History Register，GHR）记录所有分支的跳转历史
 - 01000100表示过去的8条跳转语句中，第2、6条发生了跳转
- 建立**模式历史表**（pattern history table，PHT）记录跳转历史
 - 用GHR的内容进行索引
 - 每个表项是一个2bits饱和计数器，记录该表项对应的历史pattern过去几次出现时，下一条分支语句的跳转情况
 - 当某个历史pattern再次出现时，使用对应表项中的2bits饱和计数器进行预测

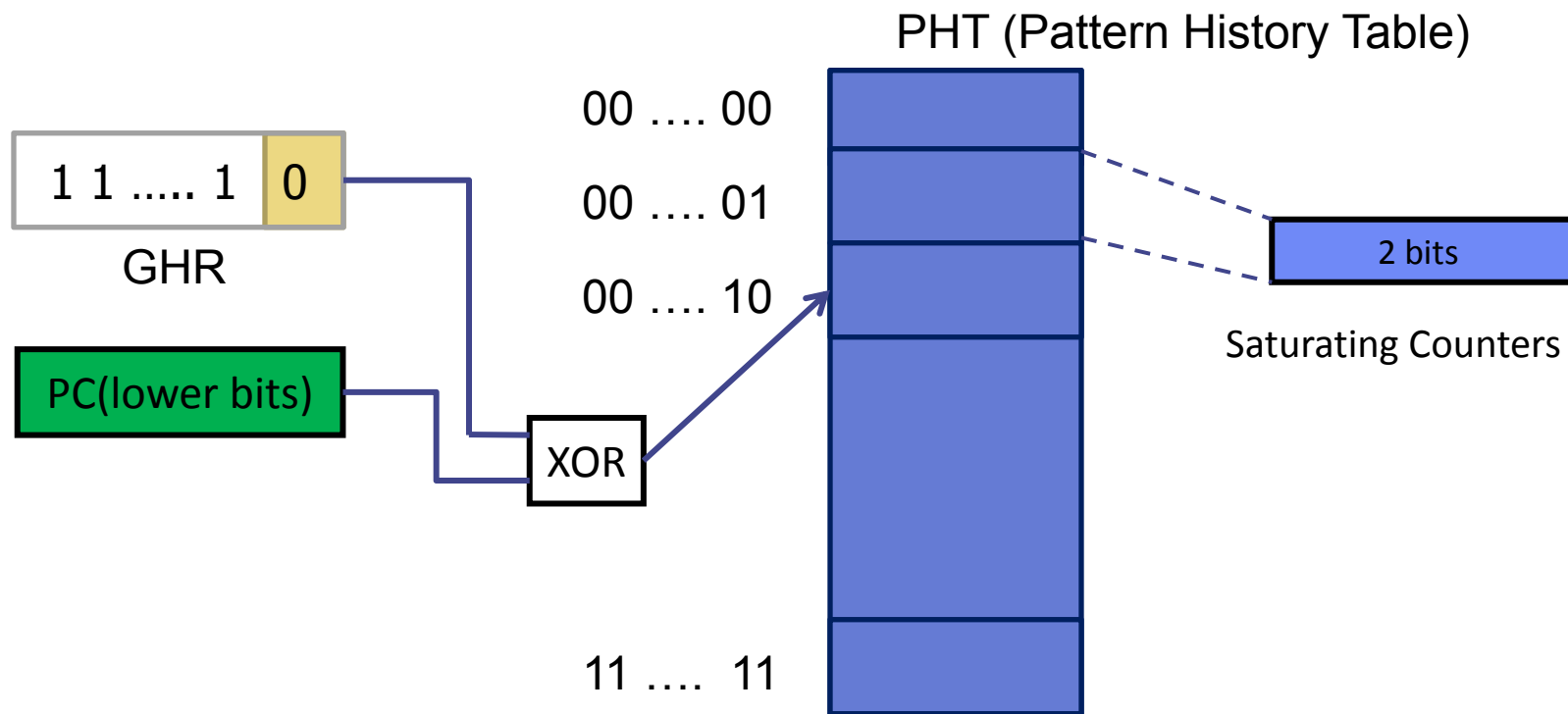
两层全局预测器

- 第一层: **全局历史寄存器** (Global branch history register , N bits)
 - 记录过去N条分支语句的跳转情况 (1 bit表示1个分支)
 - 1->taken, 0->not taken
- 第二层: **PHT的每一个表项** (对应历史pattern的饱和计数器)
 - 记录该表项对应的历史pattern过去几次出现时，下一条分支语句的跳转情况



提升全局预测器的准确率 (I)

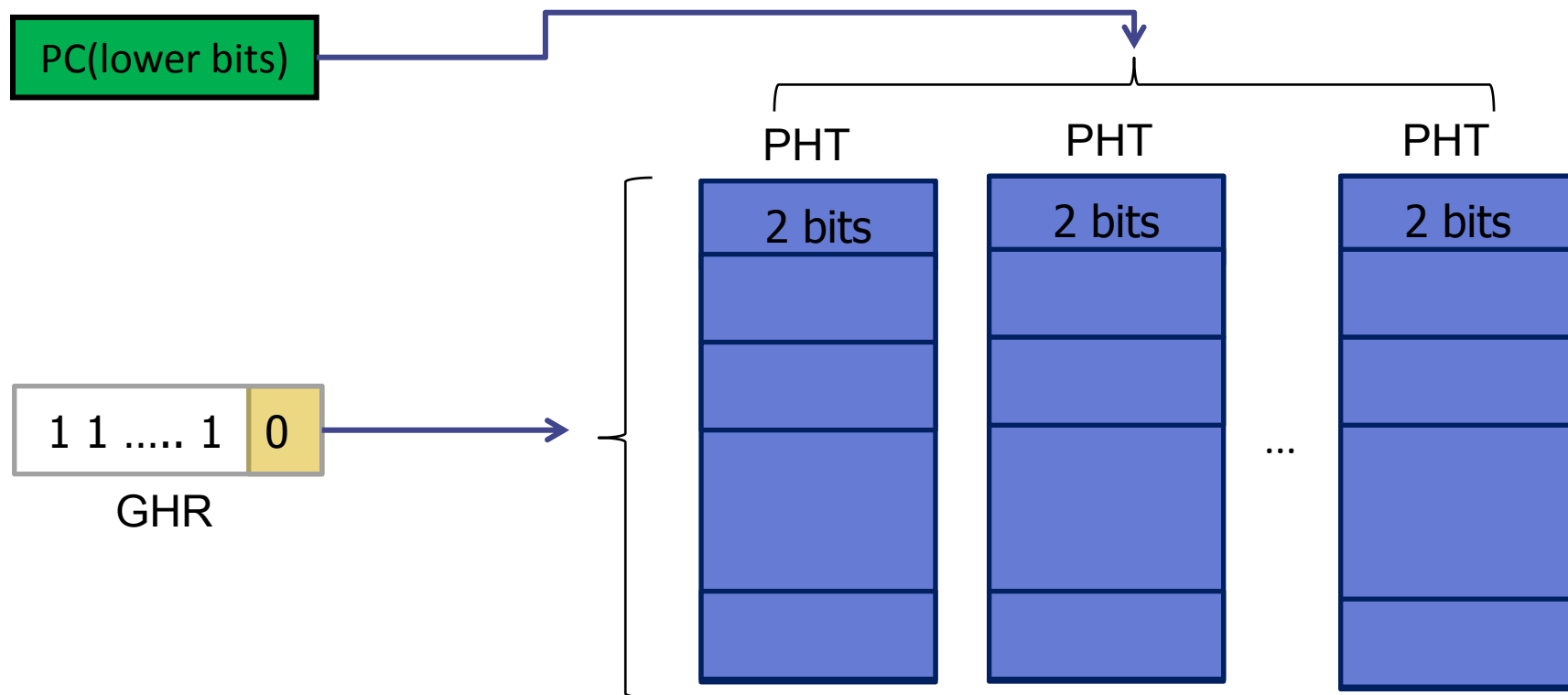
- ❑ **问题**：不同的分支语句碰到一样的历史pattern时跳转趋势可能不同
- ❑ **思路**：增加上下文信息让预测器知道当前预测是哪个分支语句
 - **Gshare predictor**: GHR和分支语句的PC进行哈希，再去索引PHT
 - + 不同的分支语句就会映射到不同的表项，不会相互干扰
 - 增加了访问延迟



提升全局预测器的准确率 (II)

□ 思路: 为不同的分支维护单独的PHT

- 由分支语句的PC决定访问哪个PHT
- 用GHR索引PHT的表项
- + 可能产生干扰的概率更小
- 增加了硬件复杂度



两层预测器 (Two-Level Prediction)

- 无论是last time predictor还是2bits饱和计数器，都仅利用了分支语句上一次 (last time) 的结果
- 现象 1: 一条分支语句的结果可能与其他分支语句的结果相关
 - 全局相关性
- 现象2: 一条分支语句的结果可能与其过去的很多次结果相关 (不仅仅是最后一次)
 - 局部相关性

局部相关性

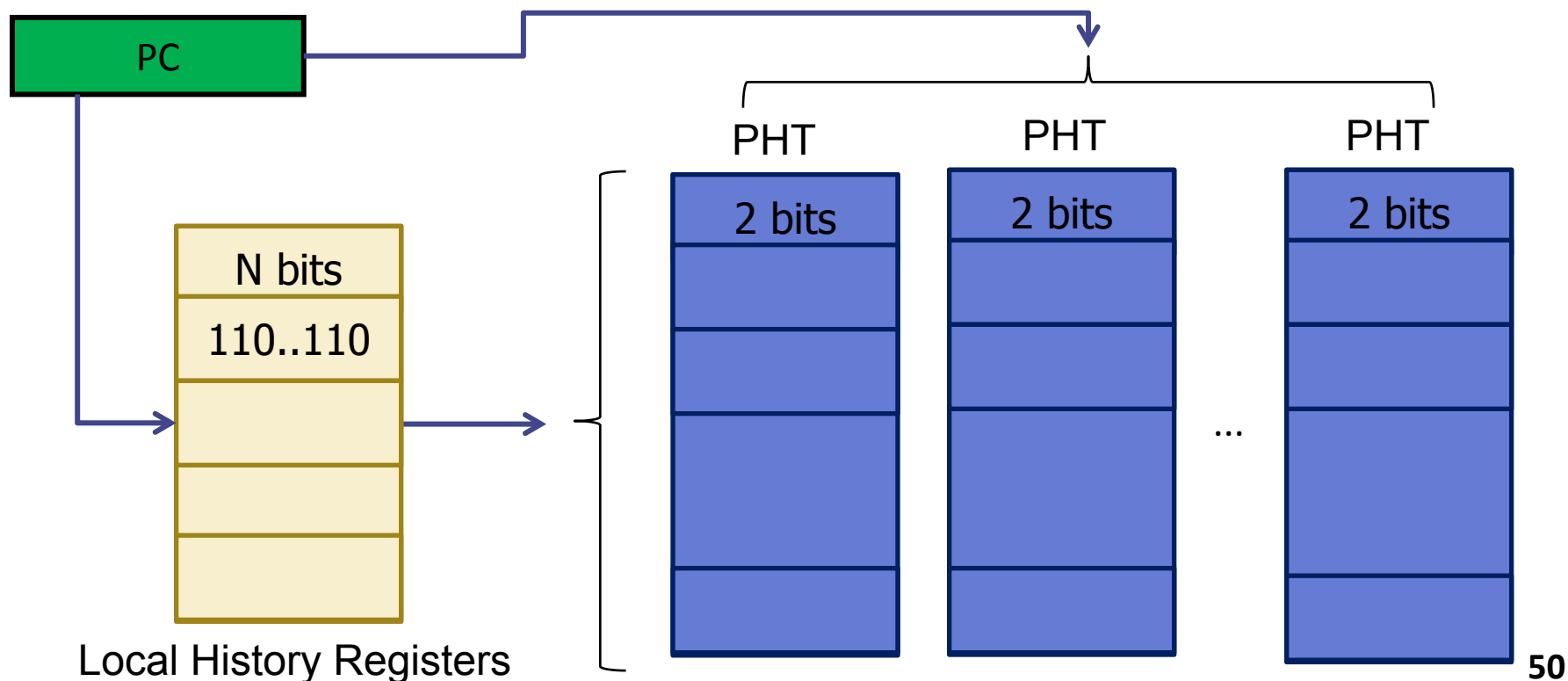
```
for (int i=1; i<4; i++)
```

Pattern: TTT**N**TTT**N**TTT**N**TTT**N**TTT**N**TTT**N**

假如我们知道该段代码的前几次分支跳转规律，就能准确预测后面的跳转情况

两层局部预测器

- **第一层:** 一组**局部历史寄存器** (local history registers , 每个N bits)
 - 历史寄存器用分支语句的PC进行索引
- **第二层:** 一组**PHT** (每个PHT的每个表项是2bits饱和计数器)
 - PC决定访问哪个PHT
 - 局部历史寄存器的值决定访问PHT的哪个表项



能否做的更好?

- 不同的分支语句特点不同
 - 有些用局部历史信息预测更好
 - 有些用全局信息预测更好
 - 有些用饱和计数器预测就能满足
 - 有些可能用1bit预测就能满足
- **结论: 没有哪个预测器适用于所有分支语句**
- **思路: 设计异构混合预测器**

混合预测器

□ **思路:** 用多种预测器同时预测，采用最好的预测结果

□ **优点:**

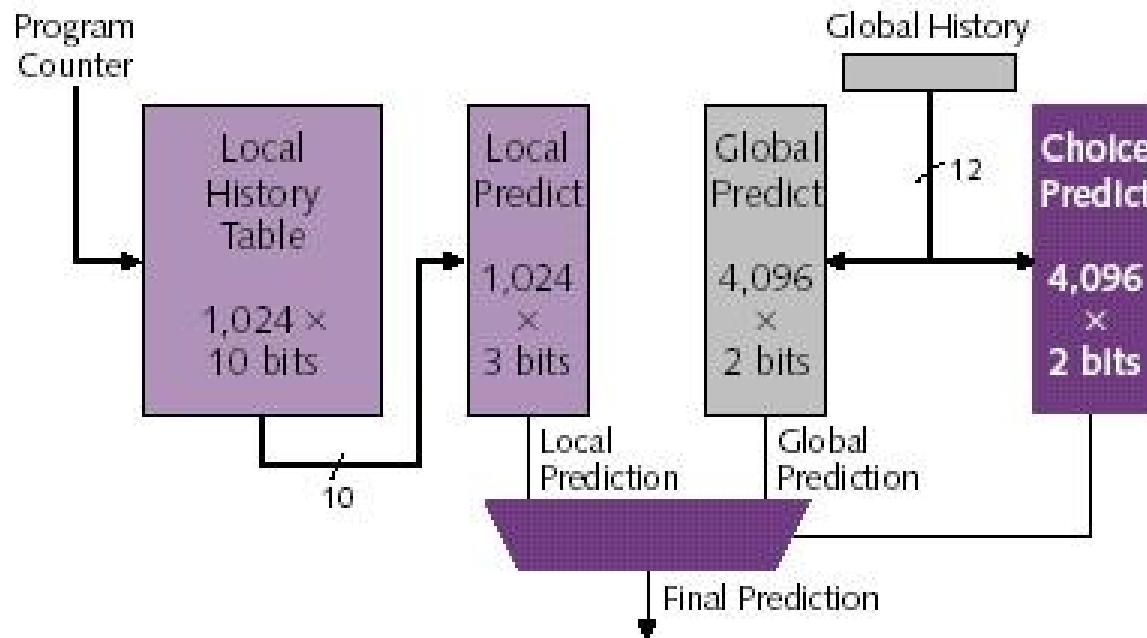
- + 准确率更高: 不同分支语句都能找到适合它的预测器

□ **缺点:**

- 需要增加选择器（选哪一个结果）
- 更复杂，延迟更高

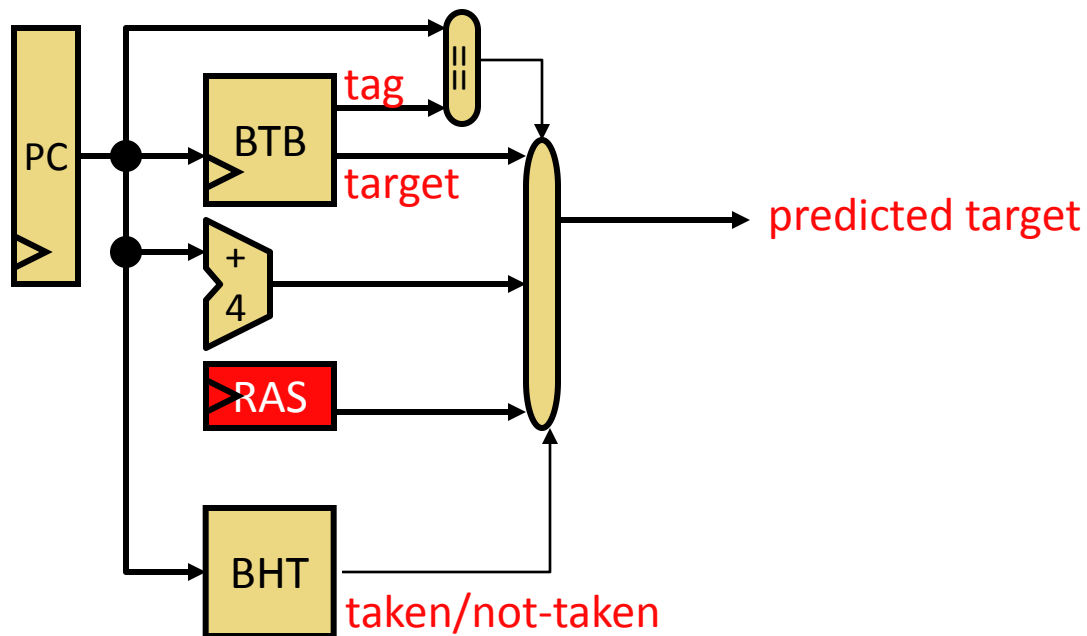
- McFarling, “Combining Branch Predictors,” DEC WRL Tech Report, 1993.

Alpha 21264 Tournament Predictor



- 最小分支惩罚: 7 cycles
- 平均分支惩罚: 11+ cycles
- 局部预测和全局预测相结合
- Kessler, "The Alpha 21264 Microprocessor," IEEE Micro 1999.

完整的分支预测



- BHT预测是否taken，如果NO，就用PC+4，否则用预测的target
- BTB判断是否是分支指令，如果是，预测target在哪里
- 如果是return，那么通过RAS栈来预测targets

预测错误恢复

(Wrong Prediction Recovery)

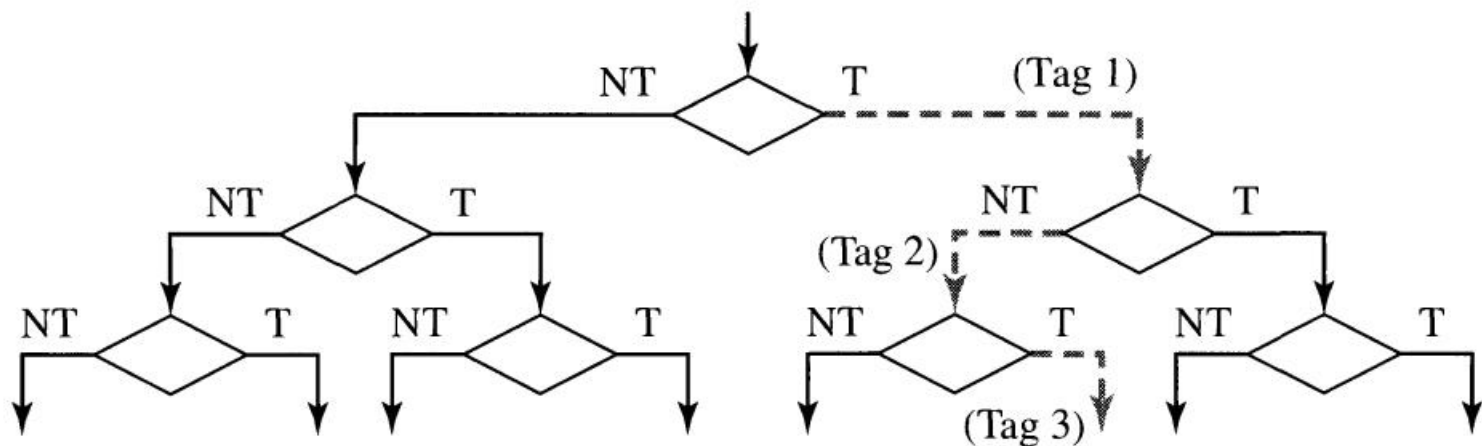
预测错误恢复

分支预测可以看作两个部分

- 前面部分->负责预测
- 后面部分->负责纠错

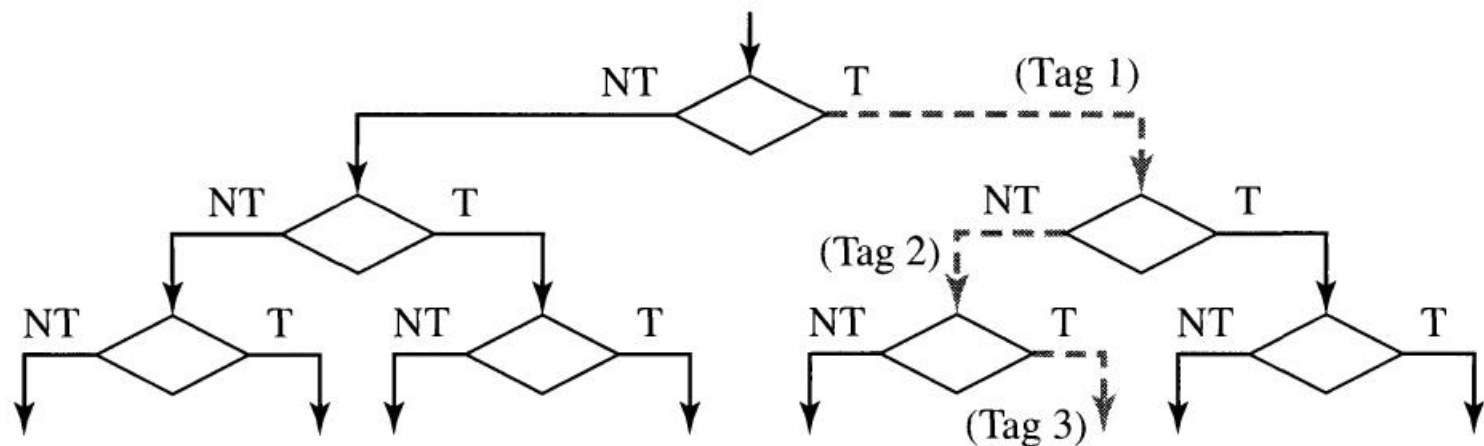
分支预测可能嵌套

- 在第一个预测没有确认结果前，又有几次预测



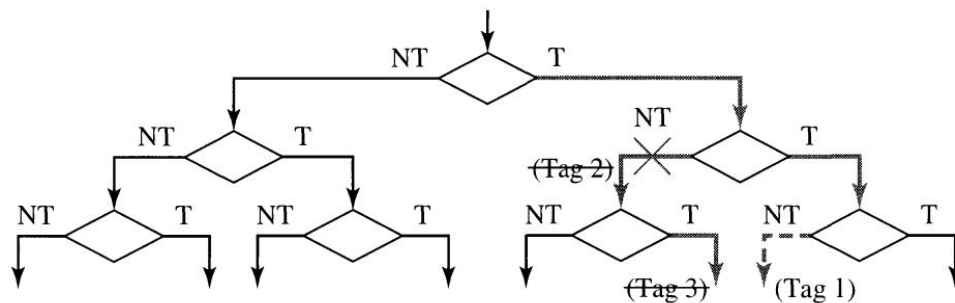
预测错误恢复

- 所有预测的分支都在系统中，并且**在decode阶段**都已经识别出是分支语句
- 同一个分支语句中的代码块，标记上相同的tag
 - 不同的分支语句，tag不同
- 有tag的指令表明该指令为推测执行
 - tag的值代表其所在的分支语句



预测错误恢复

- **分支验证在分支结果计算出来时进行**（可能在不同阶段）
 - 此时可以确定分支预测是否正确
- **如果分支预测正确**，带有该分支tag的所有指令都将去掉tag，允许提交
- **如果分支预测错误**，要做两件事
 - **终止错误的分支**
 - decode, dispatch, 保留站, ROB中所有带有该tag(以及后续tag)的指令全部清除
 - **开始正确的分支**
 - 新的分支地址根据buffer中的指令地址计算



分支预测性能分析

□ 动态预测方法

- 20%的分支指令
- 简单预测器: 准确率75%
 - $\text{CPI} = 1 + (20\% * 25\% * 2) = 1.1$
- 复杂预测器: 准确率95%
 - $\text{CPI} = 1 + (20\% * 5\% * 2) = 1.02$

□ 分支预测错误仍然是个大问题

- 流水线较深: 预测错误惩罚 10+ cycles

分支预测的未来

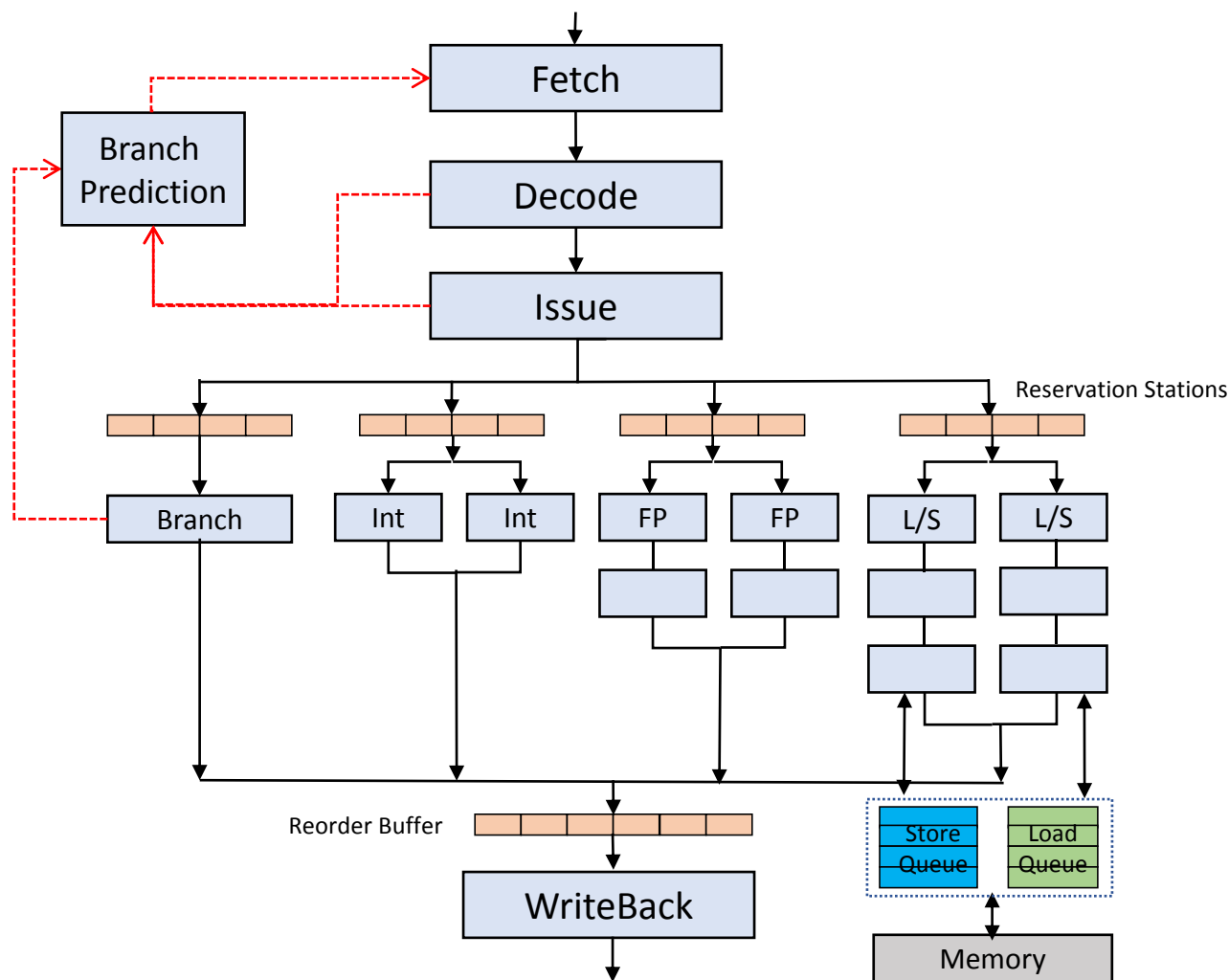
□ 提升准确率比较困难

- 95%~98%

□ 新的应用和新的架构需要新的分支预测方法

- power aware
- SMT
- neural network

截至目前，流水线：



➤ 单发射

➤ 乱序执行

➤ 有数据相关性

➤ 无分支指令

➤ 多周期指令

➤ Cache